

Getting Started with Pelee and Rondeau

Fluids Lab

October 4, 2026

Contents

1	Introduction	6
1.1	Why Are We Using Servers?	6
1.2	Important Concepts	6
1.3	Learning Objectives	6
2	Connecting to a Server	7
2.1	What is SSH?	7
2.2	Windows	7
2.3	macOS and Linux	7
2.4	First Login	7
2.5	Successful Login	7
2.6	Connecting from Off Campus (VPN)	7
2.7	Logging Out	7
2.8	Exercise	8
3	Linux Navigation	9
3.1	Where Am I?	9
3.2	Listing Files	9
3.3	Changing Directories	9
3.4	Creating Directories	9
3.5	Useful Shortcuts	9
3.6	Exercise	9
4	Files and Directories	10
4.1	Creating Files	10
4.2	Copying Files	10
4.3	Moving and Renaming Files	10
4.4	Deleting Files	10
4.5	Working with Directories	10
4.6	Exercise	10
5	Editing Files	11
5.1	Using Nano	11
5.2	Saving Changes	11
5.3	Exiting	11
5.4	Viewing a File	11

5.5	Exercise	11
6	Running Programs	12
6.1	Python	12
6.2	Julia	12
6.3	Useful Commands	12
7	Conda Environments	13
7.1	Creating an Environment	13
7.2	Activating an Environment	13
7.3	Installing Packages	13
7.4	Leaving an Environment	13
8	Environment Modules	14
8.1	Listing Available Modules	14
8.2	Loading a Module	14
8.3	Checking Loaded Modules	14
8.4	Unloading a Module	14
8.5	Common Examples	15
8.6	Finding Software	15
8.7	Exercise	15
8.8	A Common Mistake	15
9	Python Virtual Environments	16
9.1	Creating a Virtual Environment	16
9.2	Activating the Environment	16
9.3	Installing Packages	16
9.4	Leaving the Environment	16
9.5	When Should I Use Conda?	16
9.6	When Should I Use a Virtual Environment?	17
9.7	Exercise	17
9.8	Recommended Practice	17
10	Long-Running Jobs	18
10.1	Using tmux	18
10.2	Detach	18
10.3	Reconnect	18
10.4	Exercise	18
11	Monitoring Resources	19
11.1	CPU and Memory	19
11.2	Disk Usage	19
11.3	Exercise	19
12	GPU Computing	20
12.1	Checking GPU Usage	20
12.2	Good Practice	20
12.3	Exercise	20

13 Transferring Files	21
13.1 SCP	21
13.2 RSYNC	21
13.3 Mounting a Remote Folder with SSHFS	21
13.4 Exercise	21
14 Git Basics	22
14.1 Cloning a Repository	22
14.2 Saving Changes	22
14.3 Uploading Changes	22
14.4 Getting Updates	22
14.5 Exercise	22
15 Good Computing Citizenship	23
15.1 Storage	23
15.2 File Naming	23
16 Reproducible Research	24
16.1 Recommended Project Structure	24
16.2 Best Practices	24
16.3 Exercise	24
17 Troubleshooting	25
17.1 Command Not Found	25
17.2 Disk Full	25
17.3 Permission Denied	25
17.4 Program Crashed	25
17.5 Before Asking for Help	25
A Vim Survival Guide	26
A.1 Insert Mode	26
A.2 Return to Command Mode	26
A.3 Common Commands	26
A.4 Most Important Command	26
B File Permissions	26
B.1 Making a Script Executable	27
B.2 Useful Commands	27
C SSH Keys	28
C.1 Create a Key Pair	28
C.2 Waterloo Servers (Pelee and Rondeau)	28
C.3 Alliance Clusters	28
C.4 GitHub	28
C.5 A Shortcut File	29
C.6 If It Does Not Work	29

D	VSCode Remote SSH	30
D.1	Install	30
D.2	Connect	30
D.3	Benefits	30
E	Jupyter Notebooks	31
E.1	Start Jupyter	31
E.2	Create an SSH Tunnel	31
E.3	Open in a Browser	31
F	Alliance Clusters	32
G	Pelee and Rondeau Cheat Sheet	33
H	Multi-GPU Computing with Julia	34
H.1	Loading Required Modules	34
H.2	Installing Required Julia Packages	34
H.3	Testing CUDA	35
H.4	Configuring MPI.jl	35
H.5	Verifying GPU Assignment	35
H.6	Selecting GPUs	36
H.7	Checking Available GPUs	36
H.8	Checking Which GPU Julia Is Using	36
I	Multi-GPU Computing with Python	37
I.1	Loading Required Modules	37
I.2	Installing Required Packages	37
I.3	Testing CUDA	37
I.4	Verifying GPU Assignment	38
I.5	Selecting GPUs	38
I.6	MPI with Python	38
I.7	Multi-GPU PyTorch	39
I.8	Troubleshooting	39
J	Multi-GPU Oceananigans on Alliance Clusters	40
J.1	Before You Start	40
J.2	Get the Code	40
J.3	The Environment	40
J.4	One-Time Setup	40
J.5	Test Jobs	41
J.6	Your Own Scripts	41
J.7	Common Problems	41
K	Contributing to This Handbook	43
K.1	Repository	43
K.2	Reporting Errors	43
K.3	Suggesting Improvements	43
K.4	Submitting Changes	43
K.5	Source Files	43

1 Introduction

1.1 Why Are We Using Servers?

Most scientific models and datasets are too large to run efficiently on a laptop. The Fluids Lab servers, **Pelee** and **Rondeau**, provide access to faster CPUs, GPUs, larger memory, and shared storage.

You may think of them as powerful Linux computers that you access remotely.

Your Laptop $\longleftrightarrow$ Pelee / Rondeau

You work from your laptop, but the calculations happen on the server.

1.2 Important Concepts

Multi-user systems. Many researchers share the same server.

CPU versus GPU. A CPU is designed for general-purpose computing. A GPU is designed for highly parallel computations.

Home directory. Every user has a home directory:

`/home/username`

Note: for historical reasons at UWaterloo Math, `/home` is an alias for `/u`, although they have the same purpose and meaning.

Persistence. Files remain on the server after you log out.

1.3 Learning Objectives

- connect to Pelee and Rondeau;
- navigate the Linux filesystem;
- transfer files;
- run Python and Julia programs;
- monitor resource usage.

2 Connecting to a Server

2.1 What is SSH?

SSH (Secure Shell) is the standard method for connecting to a remote Linux machine.

```
ssh username@pelee.math.private.uwaterloo.ca
```

```
ssh username@rondeau.math.private.uwaterloo.ca
```

2.2 Windows

Open Windows Terminal and run the SSH command.

2.3 macOS and Linux

Open a terminal and run the SSH command.

Note: Nothing appears on the screen while typing your password. This is normal.

2.4 First Login

Type **yes** when asked whether you trust the host.

2.5 Successful Login

```
username@pelee:~$
```

2.6 Connecting from Off Campus (VPN)

Pelee and Rondeau are on the campus network, so from off campus you must connect to the university VPN first. If **ssh** hangs and then times out, the most common reason is that the VPN is not connected.

The university uses the Cisco Secure Client. Installation differs by operating system and changes from time to time, so follow the current instructions from IST:

<https://uwaterloo.atlassian.net/wiki/spaces/CWKB/pages/1736840329>

- Connect to the VPN *before* running **ssh**.
- In the second-password field, enter your Duo method; **push** sends a notification to the Duo Mobile app.
- You do *not* need the VPN for the Alliance clusters (Appendix F).

2.7 Logging Out

```
exit
```

2.8 Exercise

1. Connect to Pelee.
2. Run `pwd`.
3. Disconnect using `exit`.

3 Linux Navigation

Most interactions with a Linux server occur through a command-line interface.

3.1 Where Am I?

```
pwd
```

Example:

```
/home/username
```

3.2 Listing Files

```
ls
ls -l
ls -lh
```

3.3 Changing Directories

```
cd project1
cd ..
cd
```

Typing `cd` without an argument returns you to your home directory.

3.4 Creating Directories

```
mkdir project1
mkdir code data figures
```

3.5 Useful Shortcuts

Symbol	Meaning
.	Current directory
..	Parent directory
~	Home directory

3.6 Exercise

Create:

Try the following commands:

```
mkdir research
cd research
mkdir code data figures
ls
```

4 Files and Directories

Files and directories are the basic building blocks of a Linux filesystem.

4.1 Creating Files

Create an empty file:

```
touch notes.txt
```

Verify that it exists:

```
ls
```

4.2 Copying Files

Copy a file:

```
cp notes.txt backup.txt
```

The original file is unchanged.

4.3 Moving and Renaming Files

Rename a file:

```
mv notes.txt notes_old.txt
```

Move a file into another directory:

```
mv notes_old.txt data/
```

4.4 Deleting Files

Delete a file:

```
rm notes.txt
```

Warning: Deleted files cannot usually be recovered.

4.5 Working with Directories

Copy a directory and its contents:

```
cp -r project project_backup
```

Delete a directory and its contents:

```
rm -r project_backup
```

4.6 Exercise

Create a file called `notes.txt`. Copy it, rename it, and then delete the copy.

5 Editing Files

Many scientific workflows require editing scripts, configuration files, and text files.

5.1 Using Nano

Nano is a simple text editor that is available on most Linux systems.

Open a file:

```
nano notes.txt
```

If the file does not exist, it will be created.

5.2 Saving Changes

To save:

```
Ctrl+O
```

Press Enter to confirm.

5.3 Exiting

To exit:

```
Ctrl+X
```

5.4 Viewing a File

Display a file:

```
cat notes.txt
```

For longer files:

```
less notes.txt
```

Press q to quit.

5.5 Exercise

Create a file called `todo.txt`. Add three lines of text using Nano.

6 Running Programs

Most users connect to Pelee and Rondeau in order to run scientific software.

6.1 Python

Example file, `hello.py`, with contents:

```
print("Hello World")
```

Run using:

```
python hello.py
```

6.2 Julia

Example file, `hello.jl`, with contents:

```
println("Hello World")
```

Run using:

```
julia hello.jl
```

6.3 Useful Commands

```
which python  
echo $PATH
```

7 Conda Environments

Different projects often require different software versions.

7.1 Creating an Environment

```
conda create -n ocean python=3.12
```

7.2 Activating an Environment

```
conda activate ocean
```

7.3 Installing Packages

```
conda install numpy
```

7.4 Leaving an Environment

```
conda deactivate
```

8 Environment Modules

Many software packages on Pelee and Rondeau are managed using *environment modules*.

Modules allow users to select software versions without manually modifying environment variables.

8.1 Listing Available Modules

Display all available modules:

```
module avail
```

Example software available through modules includes

- CUDA
- Julia
- MATLAB
- Anaconda
- OpenMPI
- ParaView

8.2 Loading a Module

Load Julia:

```
module load julia
```

or load a specific version:

```
module load julia/1.10.8
```

8.3 Checking Loaded Modules

Show loaded modules:

```
module list
```

8.4 Unloading a Module

Unload Julia:

```
module unload julia
```

Unload all modules:

```
module purge
```

8.5 Common Examples

Load CUDA:

```
module load cuda/12.3.2
```

Load OpenMPI:

```
module load openmpi/4.1.7-with-cuda
```

Load Julia:

```
module load julia/1.10.8
```

8.6 Finding Software

Search for a package:

```
module spider julia
```

or

```
module spider cuda
```

8.7 Exercise

Load Julia and verify the version:

```
module load julia/1.10.8  
julia --version
```

8.8 A Common Mistake

Modules are usually reset when you start a new login session. If a command suddenly disappears after logging in, check whether the required module has been loaded:

```
module list
```

9 Python Virtual Environments

Python includes a built-in mechanism for creating isolated environments.

Virtual environments provide many of the same benefits as Conda and are often simpler for Python-only projects.

9.1 Creating a Virtual Environment

Create a new environment called `venv`:

```
python -m venv venv
```

This creates a directory containing an isolated Python installation.

9.2 Activating the Environment

Activate the environment:

```
source venv/bin/activate
```

Your command prompt should change to indicate that the environment is active.

9.3 Installing Packages

Use `pip` to install packages:

```
pip install numpy
```

You may verify the installation:

```
python -c "import numpy"
```

9.4 Leaving the Environment

Deactivate the environment:

```
deactivate
```

9.5 When Should I Use Conda?

Conda is often easier when working with:

- JupyterLab
- CUDA-enabled Python packages
- packages with compiled dependencies
- mixed Python and non-Python software

9.6 When Should I Use a Virtual Environment?

Virtual environments are often a good choice for:

- lightweight Python projects
- software development
- projects using only pip-installed packages

9.7 Exercise

Create a virtual environment, install NumPy, verify that it imports successfully, and then deactivate the environment.

9.8 Recommended Practice

Create a separate environment for each project.

For example:

```
arctic_project_env  
oceananigans_env  
machine_learning_env
```

Avoid installing large numbers of unrelated packages into a single environment.

10 Long-Running Jobs

Many computations run for hours or days. If your SSH connection is interrupted, the job may stop.

10.1 Using tmux

Start a tmux session:

```
tmux
```

Run your program normally.

10.2 Detach

Leave the session running:

```
Ctrl+b d
```

10.3 Reconnect

List sessions:

```
tmux ls
```

Reconnect:

```
tmux attach
```

10.4 Exercise

Start tmux, run a simple program, detach, and reconnect.

11 Monitoring Resources

Before running large jobs, monitor resource usage.

11.1 CPU and Memory

```
htop
```

or

```
top
```

11.2 Disk Usage

Available space:

```
df -h
```

Directory sizes:

```
du -sh *
```

11.3 Exercise

Determine how much free disk space is available and identify the largest directory in your project.

12 GPU Computing

Many scientific applications can use GPUs.

12.1 Checking GPU Usage

```
nvidia-smi
```

This displays:

- GPU memory usage;
- GPU utilization;
- running processes.

12.2 Good Practice

Always check whether a GPU is busy before starting large jobs.

12.3 Exercise

Run `nvidia-smi` and identify the available GPUs.

13 Transferring Files

Move files between your laptop and the server using SCP or rsync.

13.1 SCP

```
scp file.txt username@pelee.math.private.uwaterloo.ca:~
```

13.2 RSYNC

```
rsync -av data/ username@pelee.math.private.uwaterloo.ca:data/
```

13.3 Mounting a Remote Folder with SSHFS

SSHFS makes a folder on a server appear as an ordinary folder on your Linux computer, so you can open and edit remote files with local programs without copying them back and forth. These instructions are for Linux (on Ubuntu or Debian: `sudo apt install sshfs`).

```
mkdir -p ~/nibi
sshfs username@nibi.alliancecan.ca:/scratch/username ~/nibi
ls ~/nibi
fusermount -u ~/nibi          # unmount when finished
```

If you use the shortcut file of Appendix C (with **ControlMaster**), the multi-factor prompt appears only for the first connection.

- Use a mount for browsing, editing and small files. For large transfers use **rsync** or **Globus**, and do not run heavy programs on files in a mounted folder.
- Unmount when you are done. If a mount hangs after your network changes, use **fusermount -uz ~/nibi**.

13.4 Exercise

Transfer a small text file and verify the transfer using **ls**.

14 Git Basics

Git is a version control system that helps track changes to code and documents.

14.1 Cloning a Repository

```
git clone https://github.com/user/repo.git
```

14.2 Saving Changes

```
git add file.py  
git commit -m "Update analysis"
```

14.3 Uploading Changes

```
git push
```

14.4 Getting Updates

```
git pull
```

14.5 Exercise

Create a new directory called `git-practice`:

```
mkdir git-practice  
cd git-practice
```

Initialize a Git repository:

```
git init
```

Create a file:

```
echo "Hello Git" > README.txt
```

Add the file:

```
git add README.txt
```

Commit the file:

```
git commit -m "Initial commit"
```

Verify the commit history:

```
git log --oneline
```

15 Good Computing Citizenship

Pelee and Rondeau are shared resources.

- Keep directories organized.
- Delete unnecessary files.
- Do not monopolize CPUs or GPUs.
- Use descriptive file names.
- Document your work.

15.1 Storage

Home directories should only be used for storing small files like text files or code. Large data, such as binary output datasets, should be stored on the large RAID (Redundant Array of Inexpensive Disks) volumes, named 'fsys1' and 'fsys2'. Your folder on these storage volumes can be accessed on either pelee or rondeau by cd'ing to:

```
/mnt/autofs/sutton.math/fsys1/username  
/mnt/autofs/sutton.math/fsys2/username
```

Replace 'username' in the above with your UWaterloo 8-character userid.

15.2 File Naming

Avoid:

```
test  
test2  
test_final  
test_final2
```

Prefer:

```
arctic_run_2026_09_18
```

16 Reproducible Research

Good research should be easy for others (and your future self) to reproduce.

16.1 Recommended Project Structure

```
project/  
|-- code/  
|-- data/  
|-- figures/  
|-- notebooks/  
'-- manuscript/
```

16.2 Best Practices

- Use Git.
- Use Conda environments.
- Keep raw data separate from results.
- Write a README file.

16.3 Exercise

Create a project directory using the structure above.

17 Troubleshooting

17.1 Command Not Found

```
which python
```

Verify that the program is installed and available in your PATH.

17.2 Disk Full

```
df -h
```

17.3 Permission Denied

```
ls -l
```

Check file ownership and permissions.

17.4 Program Crashed

Check:

- available memory;
- available disk space;
- error messages.

17.5 Before Asking for Help

Include:

- the command you ran;
- the full error message;
- what you have already tried.

A Vim Survival Guide

Sooner or later you will encounter Vim.

Open a file:

```
vim file.txt
```

A.1 Insert Mode

Press

```
i
```

to begin editing.

A.2 Return to Command Mode

Press

```
Esc
```

A.3 Common Commands

Save:

```
:w
```

Quit:

```
:q
```

Save and quit:

```
:wq
```

Quit without saving:

```
:q!
```

A.4 Most Important Command

If you become stuck:

```
Esc :q!
```

B File Permissions

Linux controls who may read, write, and execute files.

Display permissions:

```
ls -l
```

Example:

```
-rwxr-xr-x script.sh
```

B.1 Making a Script Executable

```
chmod u+x script.sh
```

You can then run

```
./script.sh
```

B.2 Useful Commands

Give read and write permission to the owner:

```
chmod 600 file.txt
```

Make a file executable:

```
chmod 755 script.sh
```

C SSH Keys

SSH keys let you log in without typing a password each time, and they are how GitHub identifies you. A key is a pair of files: a *private* key that stays on your computer and must never be shared, and a *public* key (ending in `.pub`) that you give to servers.

C.1 Create a Key Pair

On your own computer:

```
ssh-keygen -t ed25519 -C "your_email@uwaterloo.ca"
```

Accept the default location and choose a passphrase, which protects the private key if someone gets the file. Show the public key with:

```
cat ~/.ssh/id_ed25519.pub
```

To avoid typing the passphrase repeatedly, add the key to the SSH agent (many desktops do this for you):

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

C.2 Waterloo Servers (Pelee and Rondeau)

```
ssh-copy-id username@pelee.math.private.uwaterloo.ca  
ssh username@pelee.math.private.uwaterloo.ca
```

You should no longer be asked for a password.

C.3 Alliance Clusters

The Alliance recommends installing your key through its account portal, not with `ssh-copy-id`: log in to <https://ccdb.alliancecan.ca>, open the SSH keys page, and paste the content of your *public* key. One upload works for all the Alliance HPC clusters. Whether you are still asked for your second factor depends on how the key is set up; see https://docs.alliancecan.ca/wiki/SSH_Keys and the Alliance page on multi-factor authentication.

C.4 GitHub

1. Copy your public key (`cat ~/.ssh/id_ed25519.pub`).
2. On GitHub, open Settings → SSH and GPG keys → New SSH key, paste the key, and save.
3. Test the connection:

```
ssh -T git@github.com
```

You should see a message greeting your GitHub username.

Clone with the SSH address (`git@github.com:user/repo.git`), not the `https` address. For a repository that is already cloned with `https`:

```
git remote set-url origin git@github.com:user/repo.git
git remote -v
```

On a cluster, do not copy your private key there. Create a separate key (with a passphrase) on the login node and add its public key to GitHub too; GitHub accepts several keys. Most compute nodes have no internet access, so run `git` commands on a login node.

C.5 A Shortcut File

Create `~/.ssh/config` on your computer:

```
Host pelee
    HostName pelee.math.private.uwaterloo.ca
    User username

Host nibi
    HostName nibi.alliancecan.ca
    User username
    ControlMaster auto
    ControlPath ~/.ssh/cm-%C
    ControlPersist 1h
```

Now `ssh nibi` is enough. `ControlMaster` keeps the first connection open for an hour and reuses it, so you answer the multi-factor prompt once, not for every new `ssh`, `scp` or `sshfs`.

C.6 If It Does Not Work

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519 ~/.ssh/config
ssh -v nibi
```

The `-v` option shows which keys are tried and where the connection fails.

D VSCode Remote SSH

Many users prefer VSCode instead of terminal editors.

D.1 Install

Install

- Visual Studio Code
- Remote SSH Extension

D.2 Connect

Open the command palette:

Ctrl+Shift+P

Select

Remote-SSH: Connect to Host

and connect to Pelee or Rondeau.

D.3 Benefits

- graphical file browser;
- interactive terminal;
- syntax highlighting;
- Git integration.

E Jupyter Notebooks

Jupyter allows interactive computing through a web browser.

E.1 Start Jupyter

```
jupyter lab --no-browser
```

The server will display a URL and port number.

E.2 Create an SSH Tunnel

From your laptop:

```
ssh -L 8888:localhost:8888 username@pelee
```

E.3 Open in a Browser

Open

```
http://localhost:8888
```

in your browser.

F Alliance Clusters

The skills learned on Pelee and Rondeau transfer directly to the national HPC systems of the Digital Research Alliance of Canada (the Alliance). To use them you need an Alliance account (<https://ccdb.alliancecan.ca>) and multi-factor authentication. You do *not* need the university VPN to reach them.

Cluster	Location	GPUs/node	Login node
Nibi	University of Waterloo (SHAR-CNET)	8	<code>nibi.alliancecan.ca</code>
Fir	Simon Fraser University	4	<code>fir.alliancecan.ca</code>
Rorqual	École de technologie supérieure, Montréal	4	<code>rorqual.alliancecan.ca</code>
Trillium	University of Toronto (SciNet)	4	<code>trillium-gpu.alliancecan.ca</code>

Table 1: GPU nodes (NVIDIA H100, 80 GB) on four Alliance clusters. Check the Alliance documentation for current details. Trillium’s CPU nodes use a different login node (`trillium.alliancecan.ca`), and the cluster is run differently from the others, so read its documentation before your first job.

Daily tasks remain similar:

```
ssh cd ls tmux git
```

The major difference is that large jobs must be submitted through a scheduler (Slurm).

G Pelee and Rondeau Cheat Sheet

Task	Command
Login	ssh username@pelee
Current directory	pwd
List files	ls -lh
Create directory	mkdir project
Edit file	nano file.txt
Copy file	cp old new
Move file	mv old new
Delete file	rm file.txt
Start tmux	tmux
Reconnect tmux	tmux attach
Disk usage	df -h
Directory size	du -sh *
Check GPUs	nvidia-smi
Logout	exit

H Multi-GPU Computing with Julia

Many Julia applications can use multiple GPUs simultaneously. Typical examples include:

- Oceananigans.jl
- MPI-based PDE solvers
- Distributed machine learning
- Large ensemble simulations

Multi-GPU programs typically require a CUDA-aware MPI installation. The typical workflow is:

1. Load CUDA, OpenMPI, and Julia modules.
2. Configure MPI.jl to use the system MPI installation.
3. Launch Julia using `mpiexec`.
4. Verify that each MPI rank can access a GPU.

H.1 Loading Required Modules

Load the required software:

```
module load cuda/12.3.2
module load openmpi/4.1.7-with-cuda
module load julia/1.10.8
```

Verify that the modules are loaded:

```
module list
```

Verify CUDA:

```
nvidia-smi
```

Verify MPI:

```
which mpiexec
```

H.2 Installing Required Julia Packages

The examples in this appendix use the following Julia packages:

- CUDA.jl
- MPI.jl
- MPIPreferences.jl

If they are not already installed, start Julia and run:

```
using Pkg
Pkg.add("CUDA")
Pkg.add("MPI")
Pkg.add("MPIPreferences")
Pkg.add("Oceananigans")
```

H.3 Testing CUDA

Before testing MPI, verify that Julia can see a CUDA-enabled GPU. Start Julia:

```
julia
```

Load CUDA.jl and check for GPUs:

```
using CUDA
CUDA.has_cuda_gpu()
```

Expected output:

```
true
```

You may also display information about the available GPUs:

```
CUDA.devices()
```

H.4 Configuring MPI.jl

Inside Julia:

```
using MPIPreferences
MPIPreferences.use_system_binary()
```

Restart Julia and verify:

```
using MPI
MPI.versioninfo()
```

H.5 Verifying GPU Assignment

Create a file called `test.jl`:

```
using MPI
using CUDA
MPI.Init()
rank = MPI.Comm_rank(MPI.COMM_WORLD)
println( "Rank $rank using GPU $(CUDA.device())" )
MPI.Finalize()
```

Launch four MPI ranks:

```
mpiexec -n 4 julia test.jl
```

Expected output:

```
Rank 0 using GPU CuDevice(0)
Rank 1 using GPU CuDevice(1)
Rank 2 using GPU CuDevice(2)
Rank 3 using GPU CuDevice(3)
```

H.6 Selecting GPUs

On a shared server, it is often desirable to restrict a program to a subset of the available GPUs. Before launching a computation, check current GPU usage:

```
nvidia-smi
```

You can restrict a program to specific GPUs using `CUDA_VISIBLE_DEVICES`. Use only GPU 0:

```
CUDA_VISIBLE_DEVICES=0 julia script.jl
```

Use GPUs 1 and 2:

```
CUDA_VISIBLE_DEVICES=1,2 julia script.jl
```

The selected GPUs are re-numbered internally. For example,

```
CUDA_VISIBLE_DEVICES=2 julia script.jl
```

makes physical GPU 2 appear as GPU 0 inside Julia. This is useful on shared servers where multiple users may be using GPUs simultaneously. When launching MPI jobs, the same variable can be used:

```
CUDA_VISIBLE_DEVICES=0,1,2,3 mpiexec -n 4 julia test.jl
```

Each MPI rank can then be assigned a GPU programmatically.

H.7 Checking Available GPUs

Before launching a large computation, check current GPU usage:

```
nvidia-smi
```

This allows you to identify which GPUs are currently in use.

H.8 Checking Which GPU Julia Is Using

Inside Julia:

```
using CUDA  
CUDA.device()
```

I Multi-GPU Computing with Python

Many Python applications can use multiple GPUs simultaneously.

Typical examples include:

- PyTorch
- TensorFlow
- JAX
- MPI-based scientific computing

I.1 Loading Required Modules

Load the required software:

```
module load cuda/12.3.2
module load anaconda3/2024.02.1
module load openmpi/4.1.7-with-cuda
```

Verify:

```
module list
```

I.2 Installing Required Packages

If necessary:

```
pip install mpi4py
pip install torch
```

I.3 Testing CUDA

Create a file called `test-cuda.py`:

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.device_count())
```

Run:

```
python test_cuda.py
```

Expected output:

```
True
4
```

I.4 Verifying GPU Assignment

Create a file called `test-multigpu.py`:

```
from mpi4py import MPI
import torch
comm = MPI.COMM_WORLD
rank = comm.rank
torch.cuda.set_device(rank)
print( f"Rank {rank} " f"using GPU " f"
{torch.cuda.current_device()} " )
```

Run four MPI ranks:

```
mpiexec -n 4 python test-multigpu.py
```

I.5 Selecting GPUs

A program can be restricted to specific GPUs using `CUDA_VISIBLE_DEVICES`.

Use only GPU 0:

```
CUDA_VISIBLE_DEVICES=0 python script.py
```

Use GPUs 1 and 2:

```
CUDA_VISIBLE_DEVICES=1,2 python script.py
```

This is useful on shared servers where multiple users may be using GPUs simultaneously.

I.6 MPI with Python

Many scientific Python applications use MPI through `mpi4py`.

Install:

```
pip install mpi4py
```

Create a file called `test-mpi.py`:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD

print( f"Rank {comm.rank} " f"of {comm.size}" )
```

Run:

```
mpiexec -n 4 python test_mpi.py
```

Expected output:

```
Rank 0 of 4
Rank 1 of 4
Rank 2 of 4
Rank 3 of 4
```

I.7 Multi-GPU PyTorch

PyTorch supports multiple GPUs using Distributed Data Parallel (DDP).

Check visible GPUs:

```
import torch

print(torch.cuda.device_count())
```

A typical launch command is:

```
torchrun --nproc_per_node=4 train.py
```

where one process is launched for each GPU.

I.8 Troubleshooting

Verify that Python can see the GPUs:

```
nvidia-smi
```

Verify that PyTorch detects CUDA:

```
python -c "import torch; print(torch.cuda.is_available())"
```

Expected output:

```
True
```

J Multi-GPU Oceananigans on Alliance Clusters

This appendix gets Oceananigans.jl running on the GPUs of the Alliance clusters, from a first test job to multi-node runs with CUDA-aware MPI. Everything that can change, such as module versions and MPI settings, lives in a tested repository, so the steps here stay short:

<https://github.com/francispoulin/Oceananigans-DRAC>

It was tested on Nibi and Fir in October 2026 (Table 2). New to the clusters? Read Appendix F first, and Appendix C for logging in with keys.

J.1 Before You Start

- An Alliance account, multi-factor authentication, and your supervisor’s account name (written `def-yourpi` below).
- A *minimal* `~/.bashrc`: no module load lines and no `LD_LIBRARY_PATH`. The environment script loads what it needs. Also check that `~/.modulerc` does not exist or pin an old `StdEnv`.
- Install packages on a *login node*: login nodes have internet but no GPUs, so `nvidia-smi` failing there is normal.

J.2 Get the Code

```
cd $SCRATCH
git clone https://github.com/francispoulin/Oceananigans-DRAC.git
# or, with a GitHub SSH key (Appendix C):
# git clone git@github.com:francispoulin/Oceananigans-DRAC.git
cd Oceananigans-DRAC
```

J.3 The Environment

```
source env/drac.sh
```

This loads the modules (`StdEnv/2023`, `gcc`, `openmpi`, `cuda`, `julia`) and sets the Julia *depot*, the folder where packages and compiled code are kept. The default is `$SCRATCH/julia_depot`, because home directories have small quotas. To use another folder, set `JULIA_DEPOT_PATH` *before* sourcing the script. Source it in every new shell; the repository’s job scripts do it for you. The line it prints shows the cluster, the repository and the depot, which is the first thing to check when something looks wrong.

J.4 One-Time Setup

```
bash setup/setup.sh
```

This takes 20–30 minutes. It installs the tested package versions and makes both MPI.jl and Julia’s own OpenMPI use the cluster’s CUDA-aware OpenMPI. In its output, the paths for `libmpi` must be under `/cvmfs`, not in your depot, and there should be no errors.

J.5 Test Jobs

```
sbatch --account=def-yourpi jobs/hello_cpu.sh
sbatch --account=def-yourpi jobs/hello_gpu.sh
sbatch --account=def-yourpi --nodes=1 --ntasks-per-node=2 jobs/checks.sh
```

Each job writes a `.out` file. The hello jobs end with `hello_oceananigans: OK`. `checks.sh` prints PASS or FAIL for each check and ends with `All checks passed`. It tests MPI, CUDA-aware communication between GPUs, a distributed run against a single-GPU run, and NetCDF and JLD2 output. To test a full node or several nodes, use the settings in Table 2, for example:

```
sbatch --account=def-yourpi --nodes=2 --ntasks-per-node=4 --mem=0 jobs/
checks.sh
```

J.6 Your Own Scripts

Copy `jobs/checks.sh` and replace its loop of checks with your command:

```
srun julia --project=. my_script.jl
```

Run one MPI rank per GPU (`-ntasks-per-node` equal to the GPUs you want per node, with `-gpus-per-task=h100:1`). In the Julia script, create the architecture with `arch = Distributed{GPU}()`. The scripts in `checks/` are working examples.

Cluster	GPUs/node	Full node	Status
Nibi	8	<code>-nodes=1 -ntasks-per-node=8 -mem=0</code>	tested
Fir	4	<code>-nodes=1 -ntasks-per-node=4 -mem=0</code>	tested
Rorqual	4	<code>-nodes=1 -ntasks-per-node=4 -mem=0</code>	not yet tested
Trillium	4	see its documentation	not yet tested

Table 2: Job settings by cluster (October 2026). For several nodes, increase `-nodes`.

J.7 Common Problems

Julia cannot find my packages, or the wrong Julia starts. Every new shell needs `source env/drac.sh`. Check with:

```
which julia; julia --version; echo $JULIA_DEPOT_PATH
```

You should see Julia 1.10.10 and your depot folder. Another version usually means that `module load` lines in `~/.bashrc`, or a `~/.modulerc` that sets an old default `StdEnv`, are interfering.

Precompilation fails with “could not spawn”. Two usual causes. (1) An old `LD_LIBRARY_PATH` in `.bashrc`: `echo $LD_LIBRARY_PATH` should print nothing. (2) Login-node limits: retry with fewer parallel tasks, or precompile inside an interactive allocation.

```
JULIA_NUM_PRECOMPILE_TASKS=2 julia --project=. -e 'using Pkg; Pkg.
precompile()'
salloc --account=def-yourpi --cpus-per-task=4 --mem=16G --time=1:00:00
```

A GPU job fails with “Out of GPU memory” on a tiny problem. The GPU may be occupied by another process or in a bad state. Look at the line printed by `hello_gpu.sh` for the memory in use at the start. Rerun; if it fails again on the same node, send the node name and job ID to Alliance support.

A job stays pending.

```
squeue -u $USER --start
```

shows the estimated start. Whole-node requests wait longer than small ones.

My packages disappeared. Scratch is temporary storage, and unused files are eventually deleted. If your depot is gone, source the environment and rerun `bash setup/setup.sh`.

K Contributing to This Handbook

This handbook is maintained through a public GitHub repository.

Suggestions, corrections, and improvements are welcome.

K.1 Repository

The source files for this handbook are hosted on GitHub:

`https://github.com/francispoulin/uwaterloo-fluids-computing-handbook`

K.2 Reporting Errors

If you discover:

- incorrect commands;
- broken examples;
- outdated software versions;
- typographical errors;
- unclear instructions;

please open a GitHub Issue.

Describe:

- the section containing the issue;
- the command that was executed;
- any error message;
- a proposed correction (if known).

K.3 Suggesting Improvements

Suggestions for new sections, tutorials, appendices, and examples are encouraged. Examples:

- new software packages;
- updated Julia workflows;
- Alliance cluster instructions;
- additional GPU examples.

K.4 Submitting Changes

Users familiar with Git may submit a pull request.

All contributions are reviewed before being merged.

K.5 Source Files

Do not edit the PDF directly.

All modifications should be made to the source files contained in the repository.